

Personalized Product Recommendation System - Project Report

Author: Punith (Puneeth M)

Repository: github.com/Puneeth7104/Personalized-Product-Recommendation-Model

Date: 5 October 2026

1. Problem Statement and Objective

Online shops sell far more products than a shopper can browse, so showing each person the items they are most likely to want next directly affects conversion and basket size. The objective of this project is to build a recommendation system that (a) turns shoppers' views, clicks, carts and purchases into a user-item interaction table, (b) compares a popularity baseline, a collaborative-filtering (matrix-factorisation) model, a content-based model and a hybrid, using Precision@K, Recall@K and NDCG@K on a time-based validation split, (c) handles cold start for new shoppers and new products, (d) analyses the recommendations for different shopper segments, and (e) serves the result through a Streamlit interface deployed from a GitHub repository.

2. Data

No real dataset was provided, so a synthetic store was generated with `src/recsys/data.py` (fixed random seed 42 for reproducibility): 2,000 shoppers, 600 products and 190,853 view/click/cart/purchase events over 90 days (1 January to 31 March 2026). Because it is synthetic it contains no confidential information and can be shared publicly. Eight hidden shopper personas (each concentrated on about six sub-categories) drive taste, so there is real collaborative signal that product metadata alone cannot fully explain. Each shopper also has a preferred price tier and a favourite brand, and stronger actions (cart, purchase) are more likely for products the shopper actually likes.

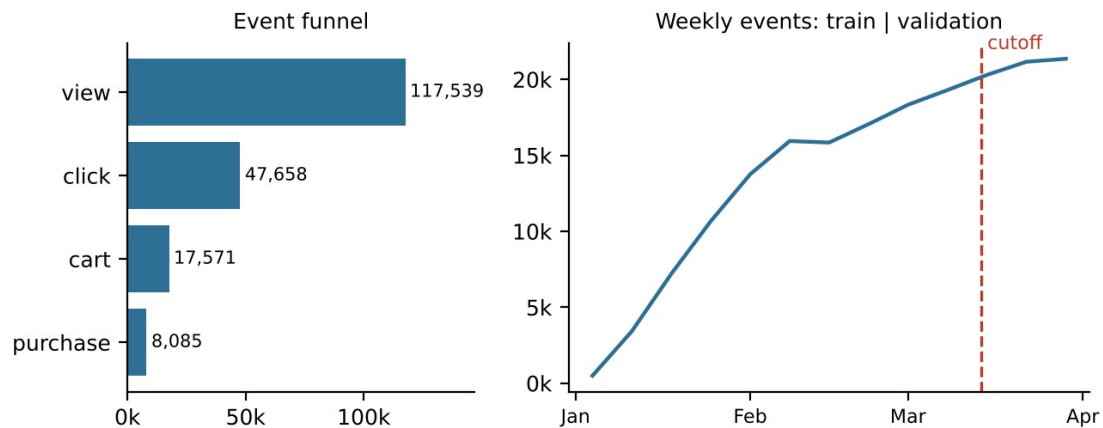
Data groups collected: the product catalogue (8 categories, 35 sub-categories, brand, three descriptive tags, price and launch day) and the event log (shopper, product, event type, timestamp). Product popularity follows a long-tailed distribution and new arrivals get a short-lived popularity boost.

Cold-start situations were built in on purpose so they can be evaluated: about 10% of shoppers sign up only after the training/validation cutoff, and about 15% of products launch in the last few days before it (or on it), so they have little or no interaction history.

3. Exploratory Findings

- The behaviour funnel narrows as expected: 117,539 views, 47,658 clicks (40.5% of views), 17,571 carts (36.9% of clicks) and 8,085 purchases (46.0% of carts).
- The time-based split is at 14 March 2026: 136,442 events are used for training and 54,411 events in the following 18 days for validation.
- Shopper activity before the cutoff is uneven: 225 of 2,000 shoppers (11%) have no history, 136 are light (1-4 distinct products), 644 medium (5-19) and 995 heavy (20+).

- 71 of 600 products (12%) are cold, meaning fewer than 5 distinct shoppers interacted with them before the cutoff.



4. Data Preparation

- Validation: the event log is checked for required columns and known event types, timestamps are parsed, and nulls and duplicate rows are dropped (function `validate_events`, which also lets real data be plugged in).
- Implicit feedback: each event type gets a weight (view 1, click 2, cart 4, purchase 8). Weights are summed per shopper-product pair and compressed with $\log(1 + \text{weight})$ so that one shopper viewing a product many times cannot dominate the model. The result is a sparse shoppers x products matrix.
- Time-based split: training uses only events strictly before the cutoff and validation only events after it. A random split would let the model learn from the future and overstate quality; a unit test checks that no training event is at or after the cutoff.
- Ground truth: for each shopper, the products added to cart or purchased in the validation window, excluding products they had already purchased in training. 1,378 shoppers have at least one such product and form the evaluation set. Views and clicks are too cheap and noisy to count as success.
- Already purchased products are never recommended, but viewed or carted products can reappear, which mirrors real retargeting.
- Leakage control: the interaction matrix, product statistics, TF-IDF vocabulary, price tiers and shopper segments are all derived from training data only.

5. Feature Engineering

- metadata tokens: category, sub-category (counted twice because it is the most informative field), brand, three tags and the price tier of each product, turned into a TF-IDF vector (sublinear term frequency, L2 normalised).
- price_tier: budget, mid or premium, defined by per-category price terciles. Because it only needs the learned cut points, a brand-new product can be assigned a tier from its price alone.
- shopper taste profile: the strength-weighted average of the TF-IDF vectors of the products the shopper interacted with, normalised. Scoring a product is a dot product with this profile.
- trending score: event weights decayed with a 14-day half-life, so recent activity counts more than old activity.

- **activity_segment**: number of distinct products in training (0, 1-4, 5-19, 20+). The same value drives the hybrid's weights and the segment analysis.
- **cold_item** flag: fewer than 5 distinct training shoppers, used by the hybrid and by the cold-item evaluation.

6. Modelling Approach

Five rankers share one training context and the same split: (1) all-time popularity, the simplest baseline; (2) trending popularity with a 14-day half-life; (3) matrix factorisation, a truncated SVD with 32 latent factors on the log-weighted shopper-product matrix, where a shopper's score for a product is the reconstruction of their row (products liked by similar shoppers rise); (4) a content-based model using the TF-IDF shopper profile from Section 5, which can score products with no interaction history; and (5) a hybrid that blends trending popularity, matrix factorisation and content scores (each min-max scaled) with weights that depend on how much history the shopper has:

Distinct products in history	Matrix factorisation	Content-based	Trending popularity
0 (new shopper)	0.00	0.00	1.00
1-4	0.20	0.50	0.30
5-19	0.45	0.40	0.15
20 or more	0.55	0.35	0.10

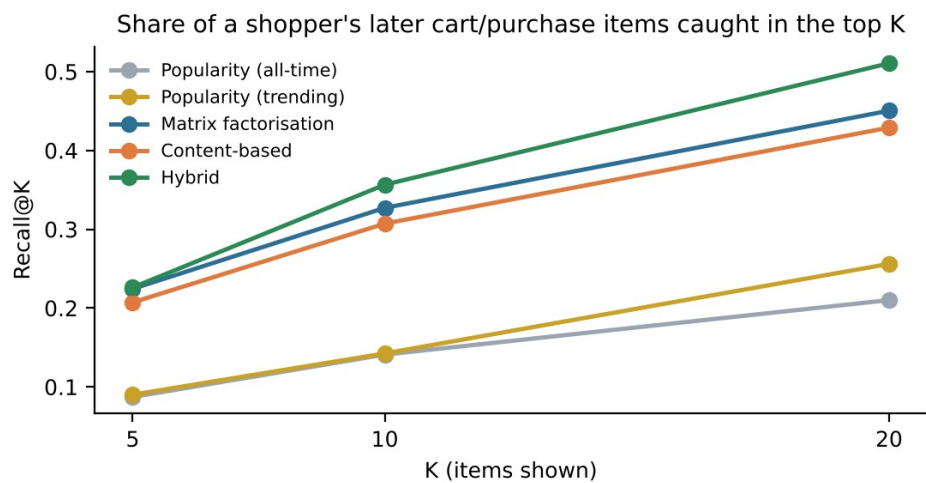
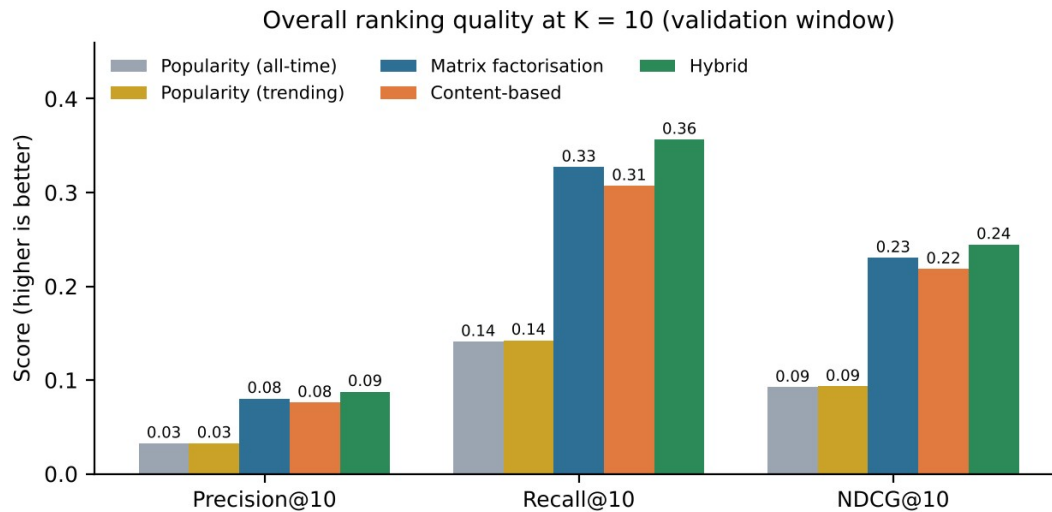
For cold products (fewer than 5 training shoppers) the matrix-factorisation slot is filled with the content score, because factorisation has nothing to say about a product nobody has touched. The weights are fixed, documented heuristics chosen in advance, not tuned on the validation window, since tuning and reporting on the same window would inflate the results. Evaluation uses Precision@K (share of the K shown products that are correct), Recall@K (share of the shopper's later cart/purchase products found in the top K), NDCG@K (like precision, but a correct product at rank 1 counts more than at rank 10) and hit rate@K (shoppers with at least one correct product).

7. Results and Model Comparison

Metrics on the validation window (1,378 shoppers, ranking all 600 products, K = 10):

Model	Precision@10	Recall@10	NDCG@10	Hit rate@10
Popularity (all-time)	0.033	0.141	0.092	0.283
Popularity (trending)	0.033	0.142	0.093	0.279
Matrix factorisation (SVD)	0.080	0.327	0.230	0.545
Content-based (TF-IDF)	0.076	0.307	0.218	0.515
Hybrid	0.088	0.357	0.245	0.581

The hybrid obtained the best score on every metric (NDCG@10 0.245) and was selected as the final model. All three personalised models are far above the popularity baselines, at about 2.3 to 2.6 times the NDCG@10 of trending popularity. In practice a hybrid top-10 list catches 36% of the products a shopper later adds to cart or buys, against 14% for all-time popularity. Matrix factorisation is the strongest single model and the content-based model is close behind, which is expected because the simulated taste has both a collaborative and a metadata component. Trending popularity barely improves on all-time popularity at K = 10 (NDCG@10 0.093 vs 0.092) but helps at larger K (Recall@20 0.256 vs 0.210).



How big is the hybrid's advantage? On the default seed it is +0.014 NDCG@10 over matrix factorisation. To check that this is not a one-seed accident, the whole pipeline (data, split, models) was repeated on five other random seeds:

Seed	Trending popularity	Matrix fact.	Content-based	Hybrid	Hybrid - MF
42 (default)	0.093	0.230	0.218	0.245	+0.014
1	0.077	0.202	0.196	0.218	+0.016
2	0.080	0.243	0.205	0.250	+0.007
3	0.069	0.214	0.175	0.207	-0.007
4	0.094	0.236	0.183	0.250	+0.015
5	0.110	0.229	0.187	0.237	+0.008
Mean of seeds 1-5	0.086	0.225	0.189	0.232	+0.008

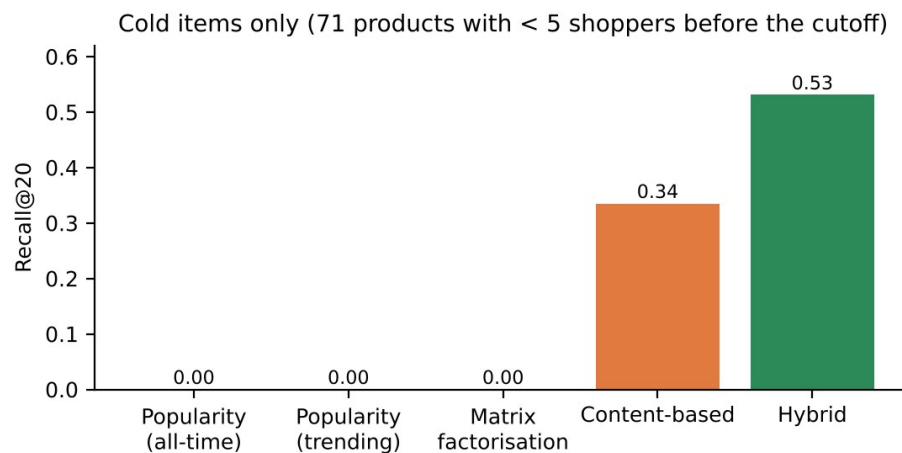
Personalised models beat popularity on every seed, and matrix factorisation beat the content-based model on all five. The hybrid beat matrix factorisation on 4 of the 5 other seeds, by 0.008 NDCG@10 on average, and lost on one. The honest summary is that the hybrid is modestly better than matrix factorisation overall,

and its clear advantage is in the cold-start cases below. No confidence intervals were computed, so the small gap should be treated as suggestive.

Cold products. 183 of the evaluated shoppers added a cold product (fewer than 5 training shoppers) to cart or bought one. Scoring only those products:

Model	Recall@10	Recall@20	NDCG@20
Popularity (all-time)	0.000	0.000	0.000
Popularity (trending)	0.000	0.000	0.000
Matrix factorisation (SVD)	0.000	0.000	0.000
Content-based (TF-IDF)	0.206	0.335	0.159
Hybrid	0.457	0.531	0.302

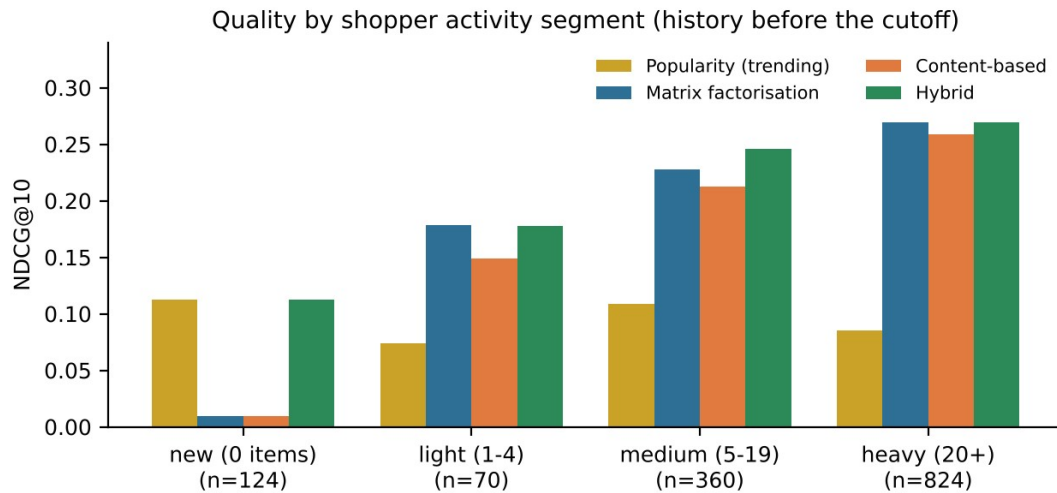
Popularity and matrix factorisation score exactly zero here: they can only recommend products somebody has already interacted with. The content-based model reaches Recall@20 of 0.34 using metadata alone, and the hybrid reaches 0.53 because it combines that with collaborative and trending signals for the rest of the catalogue. Across the five other seeds the hybrid's cold-item Recall@20 averaged 0.54 (range 0.46 to 0.62), while matrix factorisation stayed at 0.00 on every seed.



8. Segment Analysis

Shoppers are segmented using training data only, by activity (number of distinct products), dominant category and price level. NDCG@10 by activity segment for the evaluated shoppers:

Segment	Shoppers	Trending pop.	Matrix fact.	Content-based	Hybrid
new (0 items)	124	0.112	0.009	0.009	0.112
light (1-4)	70	0.074	0.179	0.149	0.178
medium (5-19)	360	0.109	0.227	0.213	0.246
heavy (20+)	824	0.085	0.269	0.258	0.269



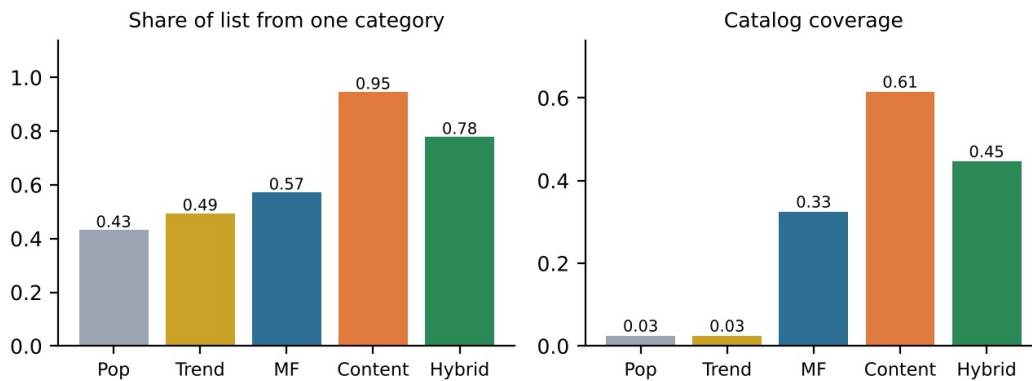
- New shoppers (no history): matrix factorisation and the content model have nothing to personalise on and return arbitrary lists (NDCG@10 0.009). The hybrid detects the empty history and shows trending products, matching the best option (0.112). Plain matrix factorisation would be about 12 times worse for this group.
- Light shoppers (1-4 products): personalised models already beat popularity by a wide margin (0.179 and 0.178 vs 0.074), and the hybrid is level with matrix factorisation.
- Medium shoppers (5-19): the hybrid's best case (0.246 vs 0.227), where blending metadata with collaborative signal adds the most.
- Heavy shoppers (20+): everything personalised scores highest here (about 0.27) because there is the most evidence, and the hybrid and matrix factorisation are effectively tied.

By price segment (strength-weighted price level of past products within their category) the hybrid scores 0.246 for budget, 0.249 for mid and 0.280 for premium shoppers; the content-based model is relatively strongest for premium shoppers (0.287) and weakest for mid (0.206). These differences are small and come from a single validation window.

Quality is not the only thing that varies by model. For each segment, the top-10 lists were profiled for price, variety, novelty, share of cold products and catalogue coverage. For heavy shoppers:

Model	Mean price (\$)	Share from top category	Categories per list	Cold-item share	Catalogue coverage
Popularity (all-time)	38	0.43	3.9	0.00	0.03
Popularity (trending)	32	0.49	3.9	0.00	0.03
Matrix factorisation (SVD)	51	0.57	3.4	0.00	0.33
Content-based (TF-IDF)	55	0.95	1.2	0.04	0.61
Hybrid	54	0.78	2.1	0.16	0.45

What each model recommends to heavy shoppers (20+ items), top 10



The content-based model is the narrowest: about 95% of a heavy shopper's list comes from one category, a filter-bubble risk, although it covers the most of the catalogue across shoppers (62%). Popularity shows everyone the same few products (coverage 2.5%). Matrix factorisation is the most varied personalised model (57% from one category) and the hybrid sits in between (78%), so its accuracy gain comes with some loss of variety. A diversity re-ranker is listed under future work.

9. Deployment and Interface

The system runs as a Streamlit app (`streamlit_app.py`) with five tabs. Recommendations: pick a shopper, a model and K, see the shopper's past activity, the top-K list with a plain-language reason for each item (for example 'Similar to: ...' or 'Trending with all shoppers') and a check of how many of the shopper's actual later cart/purchase products were found. Similar and new products: products similar by metadata and by behaviour, and a cold-start demo where a new product is described (category, brand, tags, price) and the app returns its closest existing products and the shoppers most likely to be interested. Model evaluation: the metric tables at K = 5, 10 and 20, overall and for cold products. Segment analysis: the tables and profiles from Section 8. About: method and data notes.

The app generates its data and trains all models on first load (about 10 seconds), so no data or model files need to be committed. The repository is set up for Streamlit Community Cloud deployment (main file `streamlit_app.py`, dependencies in `requirements.txt`) and includes a GitHub Actions workflow that runs the 31 automated tests on Python 3.11 and 3.12 for every push to main and every pull request. The tests cover the metrics (hand-computed values), time-split leakage, the funnel and launch-date constraints, recommendation uniqueness and determinism, exclusion of purchased products, cold-start behaviour and the segment tables. Evaluation tables are saved in the reports folder (`results.md` and CSV files).

10. Limitations

- The data is synthetic, so the absolute metrics describe this simulated world, not a real shop. The simulator contains taste clusters and metadata effects, which favours the model families used here; real performance would differ. The pipeline accepts a real event log with the same columns.
- There is one train/validation window and no confidence intervals. The five-seed check shows the ranking of the models is stable, but the hybrid's small edge over matrix factorisation is not statistically tested.

- Hyperparameters are not tuned (32 latent factors, the hybrid weights and the 14-day half-life were set in advance), and truncated SVD is a simple method for implicit feedback; ALS or BPR handle it more properly.
- Only offline metrics are measured. They do not capture how showing recommendations changes behaviour, which only an A/B test can measure.
- Success is defined as cart or purchase within 18 days; a different definition (purchase only, or a longer window) would change the numbers.
- Models retrain from scratch when the app starts; there is no incremental update or stored model.

11. Future Work

- Replace the synthetic data with a real events log and re-run the pipeline.
- Implicit ALS or BPR instead of SVD; tune factors and hybrid weights on a separate validation window and report on a later test window.
- Walk-forward backtesting with confidence intervals.
- Diversity re-ranking (for example MMR) to counter the content model's narrow lists.
- A small FastAPI endpoint (/recommend/{user_id}) wrapping the same models.
- Session-aware recency features, stock and price constraints, and an A/B test to measure real uplift.

12. How to Reproduce

`pip install -r requirements.txt; python -m pytest -q; python scripts/run_pipeline.py; streamlit run streamlit_app.py`